FULLERTON HOSPITAL SYSTEM

Team :
Yamato Eguchi, Desire Hernandez,
Jong Kim, Ethan Jones, Irvin Rafael,
Saul Ruiz, & Cole Eastlund

Background

- All in one hospital management system
- Focused on automation
- Easy to use for all users
- 250 beds
- Roughly 1,000 on staff employees [1]

Development Process

Unified Process with Iterative Methods

- **Inception:** Easy to use, fast, and secure system for healthcare providers, patients, and admins
- **Elaboration:** Identify requirements and use cases
- **Construction:** Working on low level risks and implementation of easier components
- **Transition:** Conducting user tests to ensure usability and functionality

OBJECTIVES

- Patients:
  - Access to medical data and profile
  - Manage appointments
  - Medical data transfer
  - Patient and healthcare provider communication
- Admins
  - Billing
  - Manage appointments
- Doctors/Nurses:
  - Access and manage patient data
  - Schedule availability
  - Patient and healthcare provider communication
- Insurance:
  - Visibility of costs
  - Secure payment

STAKEHOLDERS

Patients

- Primary user
- System should provide ease of use

Doctors/Nurses

- Ability to create, edit, and delete patient data
- Inventory management

Admins

- Billing
- Handling appointments
- Inventory management

Insurance Companies

- Secure payment
- Bill breakdowns

REQUIREMENTS

Patient Management:

- Patient registration
- Record keeping of medical history
- Tracking admissions, discharges, transfers, appointments

Staff Management:

- Registration and management of staff
- Scheduling and task tracking
- Attendance tracking

Billing and Finance:

- Handling insurance claims and processing payments.
- Hospital finance management
- Transaction recording and tracking

Inventory & Pharmacy Management:

- Supply monitoring and updating
- Expiration date tracking
- Managing prescriptions

REQUIREMENTS

Appointment Scheduling:

- Appointment booking
- Availability management
- Reminders

Electronic Health Records:

- Maintaining patient data
- Available to both patients and providers

Report Generation & Analytics:

- Statistic and trend generation
  - Demographics
  - Treatments
  - Alements

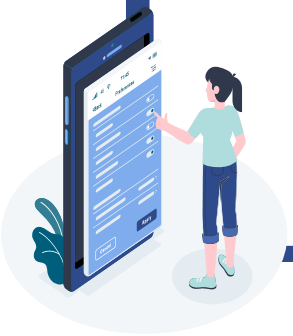
Integration & Interoperability:

- Integrating with external systems
- Ensuring interoperability with medical equipment

REQUIREMENTS

Non-Functional:

- **Availability**
  - Minimal downtime
  - Redundancy and failover mechanisms
- **Usability**
  - Intuitive and easy to use
  - Disabled friendly
  - Multi-lingual
  - Mobile use as well as desktop
- **Security**
  - Prioritize data security and integrity
- **Compliance**
  - Complies with regulations and healthcare IT system standards
  - Undergo regular audits



ARCHITECTURE

HARDWARE

- Network is on site
- Database is on site

SOFTWARE

- Database will be implemented using SQL
- Website implemented with React, HTML, CSS, and Javascript

Domain Model Chart

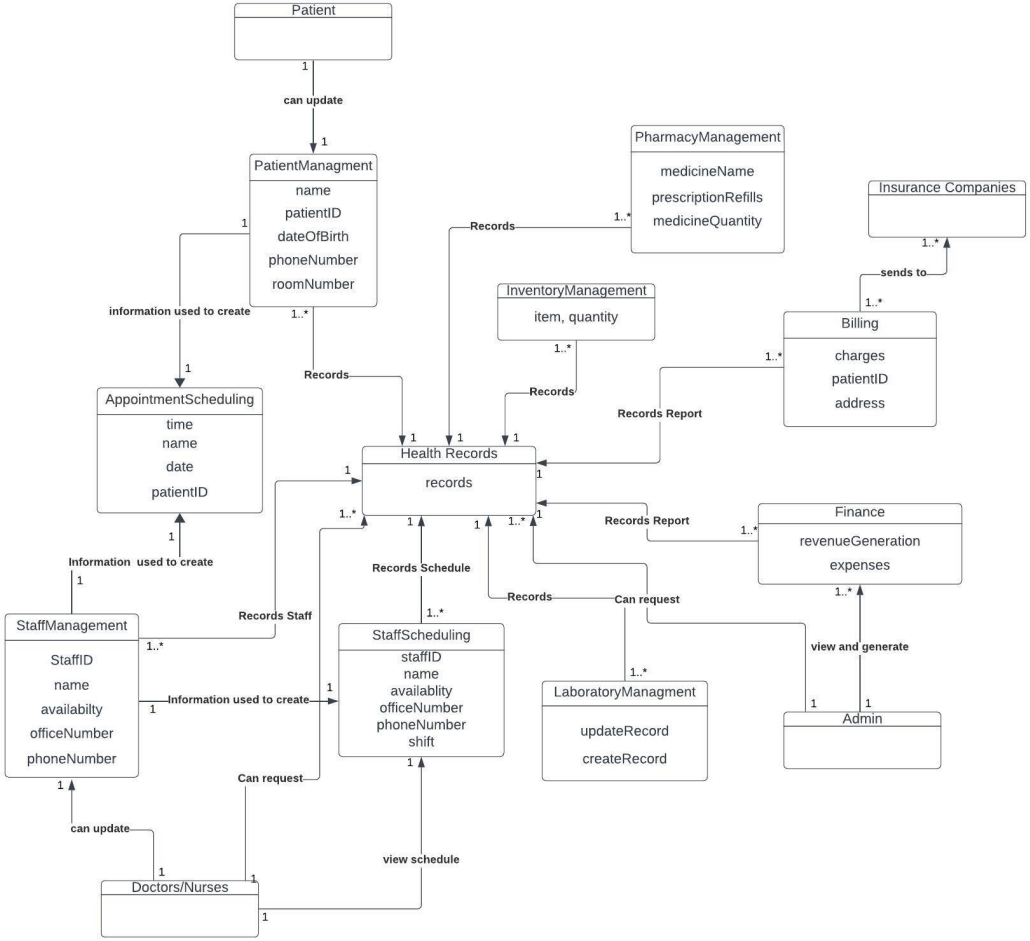
Conceptual Classes	Attributes
AppointmentScheduling	name- patient's name time- time date- date patientID- patients ID number staffID- staff members ID department- department the appointment will take place in
HealthRecords	record- includes all information about the patients and their appointments
Analytics	data- data
Integration	data - data
StaffScheduling	staffID - staff members ID name - name of the whole staff time - time availability - open time slots shift - time shift takes place for specific staff members

Domain Model Chart

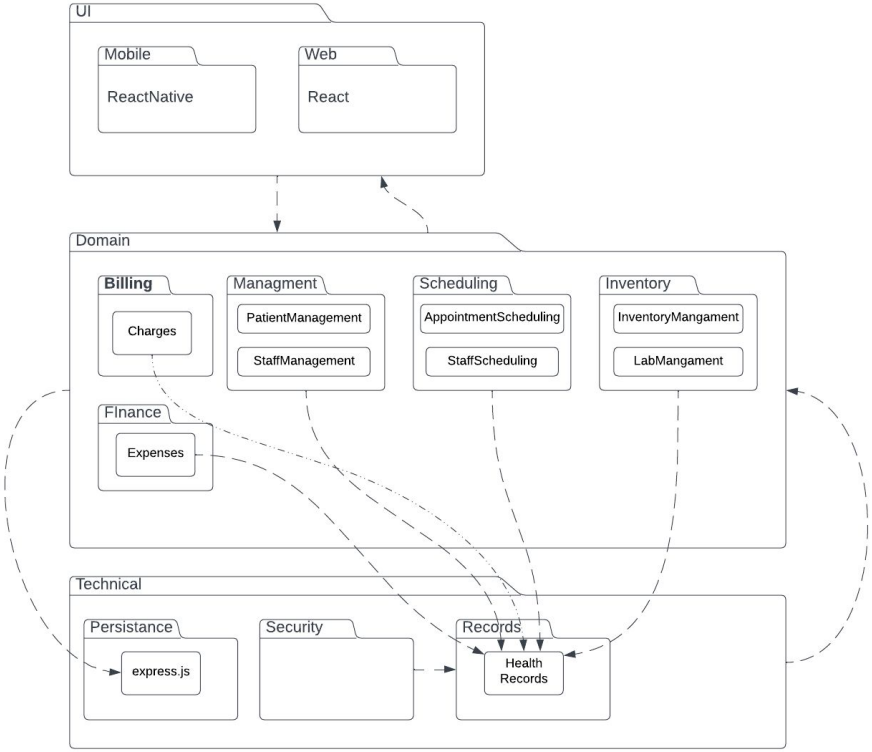
Conceptual Classes	Attributes
PatientManagement	name- name of patient patientID- patient ID number dateOfBirth- date of birth roomNumber- room number phoneNumber- patient's phone number
StaffManagement	staffID - staffs ID number name - name of staff member availability - time officeNumber - office number phoneNumber - staff members ID number
InventoryManagment	Item, quantity- name of item and quantity
Billing	charges - money charged patientID - patient ID number address - patients address

Domain Model Chart

Conceptual Classes	Attributes
Finance	revenueGeneration - amount of revenue the hospital makes expenses - the amount of money the hospital has spent
LaboratoryManagment	updateRecords- update current and past records in the system createRecords - add new records in the system
PharmacyManagment	medicineName- name of specific medicine prescriptionsRefills- prescription medicine refills medicineQuantity- amount of medicine there is


Patient
can update
PatientManagment
name
patientID
dateOfBirth
phoneNumber
roomNumber
PharmacyManagement
medicineName
prescriptionRefills
medicineQuantity
Insurance Companies
Records
sends to
InventoryManagement
item, quantity
Billing
charges
patientID
address
information used to create
Records
Records
Records Report
AppointmentScheduling
time
name
date
patientID
Health Records
records
Records Report
Finance
revenueGeneration
expenses
Information used to create
Records Schedule
Records
Can request
Records Staff
StaffManagement
StaffID
name
availabilty
officeNumber
phoneNumber
StaffScheduling
staffID
name
availablity
officeNumber
phoneNumber
shift
view and generate
Information used to create
LaboratoryManagment
updateRecord
createRecord
Admin
Can request
can update
view schedule
Doctors/Nurses

UML Package Diagram



Namespace - UI

```
namespace fullertonHospitalSystem{
    class mobile {
        //mobile
    };
    class web {
        // web
    };
}
```

Namespace - Domain

```
namespace fullertonHospitalSystem
{
    //Domain Layer
    class billing
    {
        void charge(patientinfo charge);
    };
    class management
    {
        void patientManagement(patientInfo info);
        void staffManagement(staffInfo info);
    };
    class scheduling
    {
        void appointmentScheduling(patientInfo info, staffInfo status);
        void staffScheduling(staffInfo availabilty);
    };
    class inventory
    {
        void inventoryManagement(item id);
        void labManagement(record id);
    };
    class finance
    {
        void expenses(record id);
    };
}
```

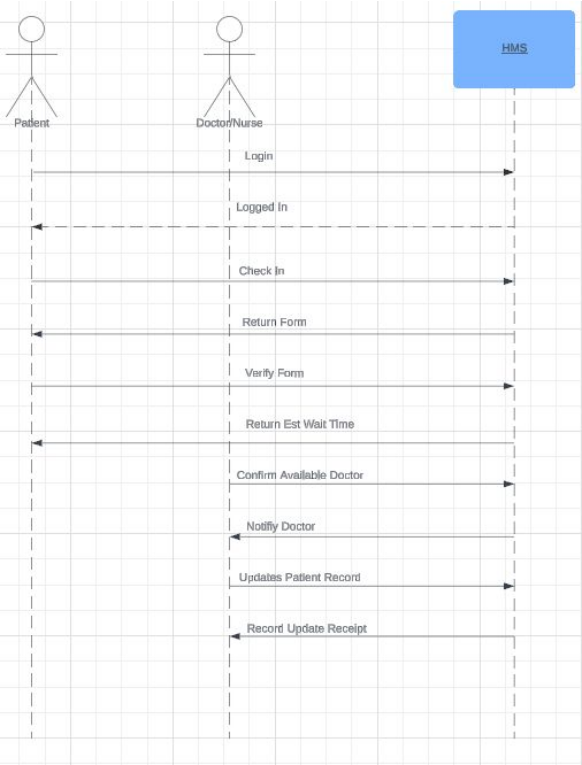
Namespace - Technical

```
namespace fullertonHospitalSystem {
    class persistance {
        //persistance
    };
    class security {
        //security
    };
    class records {
        //HR
    };
}
```

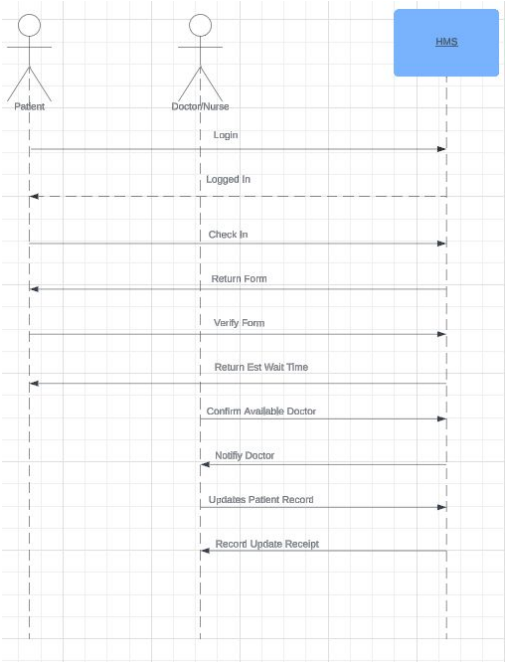
USE CASE #1- Visiting Patient

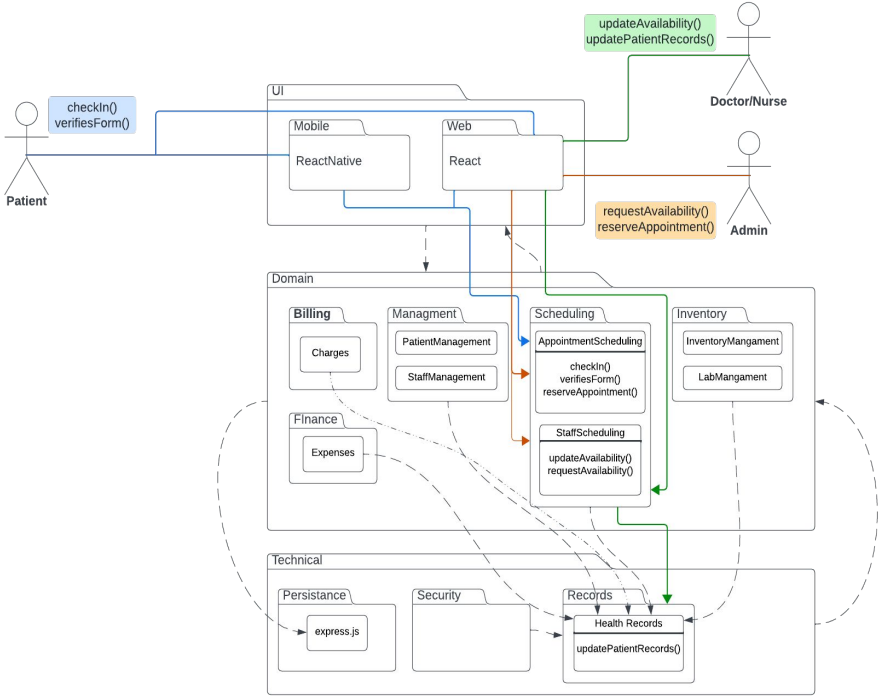
Actors: Patient, Doctor/Nurse, Administrators
Description: A **patient** arrives at the hospital for a scheduled appointment. Using the app or web portal, the **patient** checks in for their appointment. The HMS app pre-populates the check-in form with past check-in data, which the **patient** reviews and confirms for accuracy. Upon completion, the appointment scheduling system receives a notification of the **patient's** check-in and assigns a room for the visit. The **patient** undergoes medical assessment and the healthcare provider updates their **patient** record in the patient management section of the HMS. The HMS then sends the healthcare provider a receipt showing the **patient's** record was successfully updated.

USE CASE #1 - SSD



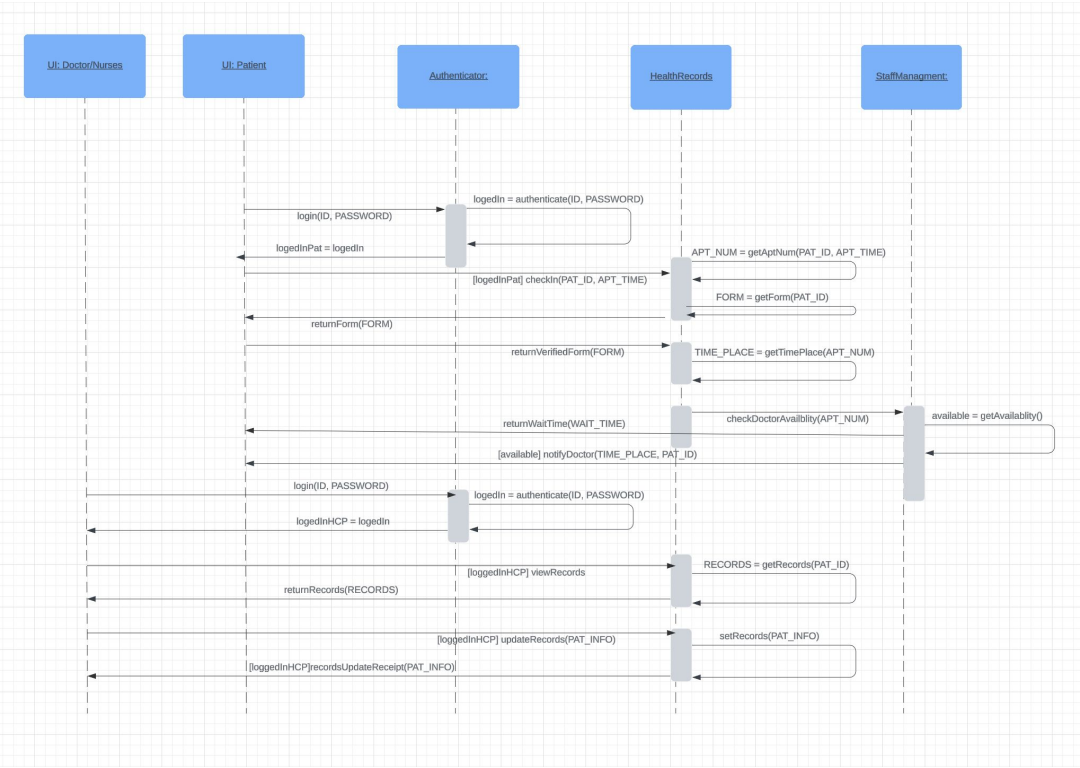
USE CASE #1 - SSD Interactions



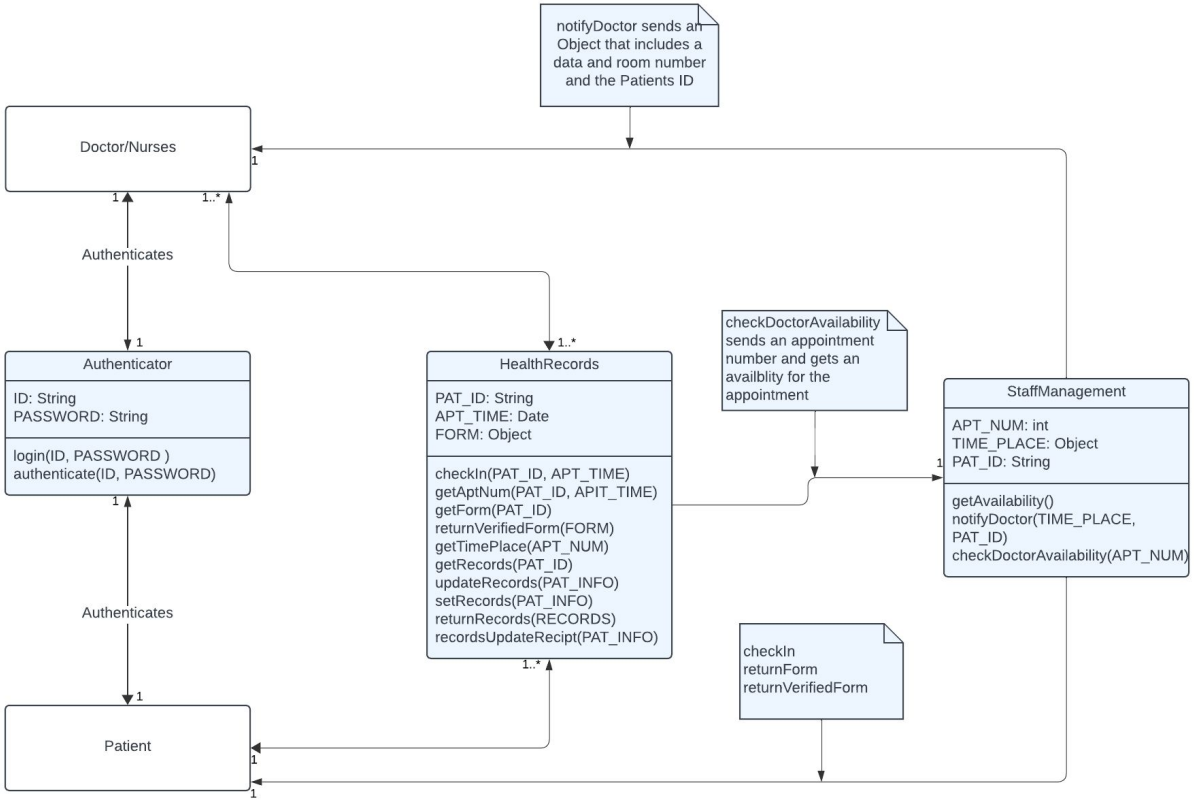


The system operations handled by the system in an SSD represent the operation calls on the Application or Domain layer from the UI layer.

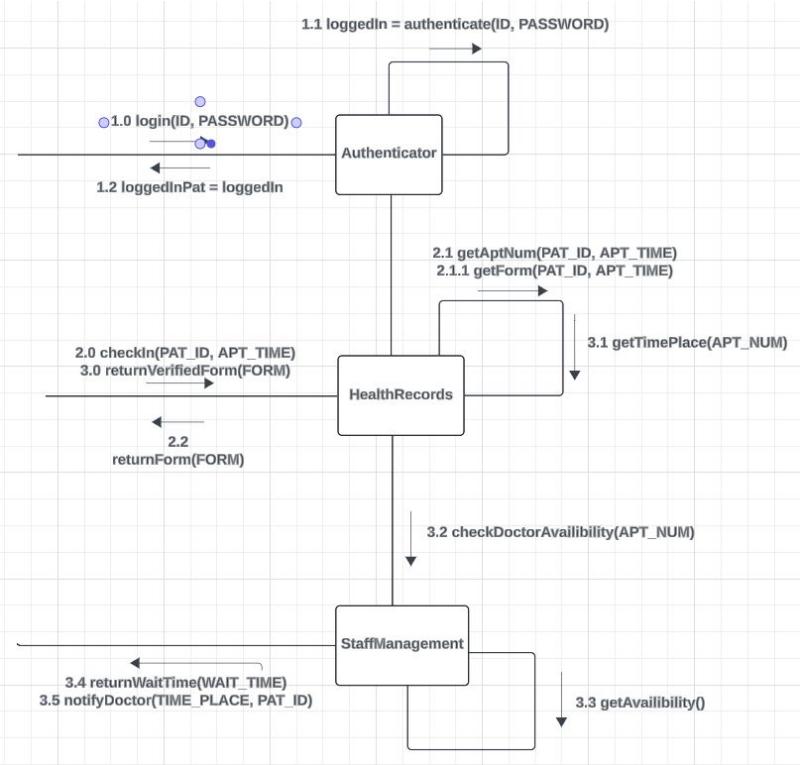
USE CASE #1 - SD

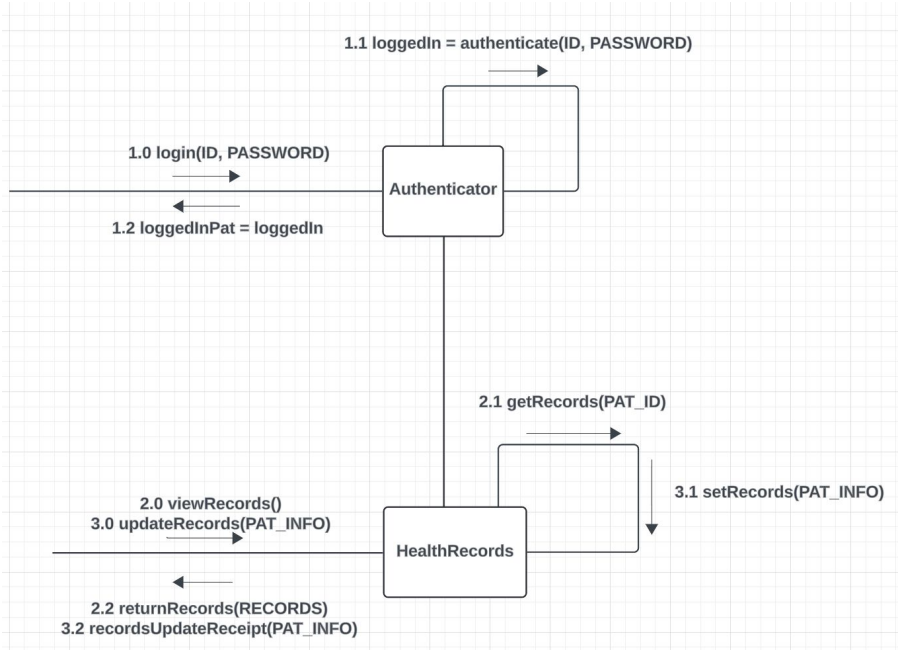

UI: Doctor/Nurses
UI: Patient
Authenticator:
HealthRecords
StaffManagment:
logedIn = authenticate(ID, PASSWORD)
login(ID, PASSWORD)
logedInPat = logedIn
APT_NUM = getAptNum(PAT_ID, APT_TIME)
[logedInPat] checkIn(PAT_ID, APT_TIME)
FORM = getForm(PAT_ID)
returnForm(FORM)
returnVerifiedForm(FORM)
TIME_PLACE = getTimePlace(APT_NUM)
checkDoctorAvailblity(APT_NUM)
available = getAvailablity()
returnWaitTime(WAIT_TIME)
[available] notifyDoctor(TIME_PLACE, PAT_ID)
login(ID, PASSWORD)
logedIn = authenticate(ID, PASSWORD)
logedInHCP = logedIn
[loggedInHCP] viewRecords
RECORDS = getRecords(PAT_ID)
returnRecords(RECORDS)
[loggedInHCP] updateRecords(PAT_INFO)
setRecords(PAT_INFO)
[loggedInHCP]recordsUpdateReceipt(PAT_INFO)

Use Case #1 - Class Diagram



USE CASE #1 - Comm/Interaction Diagrams





USE CASE #1 - Contract #1

Operation:	ReturnsPreFilledForm(patientID: integer)
Responsibilities:	Gather previous patient data from database, fill out the check-in form, and return it to the patient for review
Cross References:	Use Case 1 Visiting Patient
Exceptions:	New patient with no previous data
Pre-conditions:	An appointment exists for the patient within an hour of check-in and they have begun the check-in sequence by requesting a check-in
Post-conditions:	A check-in form has been created and pre-populated with patient data for review

USE CASE #1 - Contract #2

Operation: InformSystemOfAvailablity(Availability: boolean)

Responsibilities: When a healthcare professional puts in a notice of availability an update will be sent to the system of that availability for patient care

Cross References: Use Case 1 Visiting Patient

Exceptions: None

Pre-conditions: Healthcare professional is on shift and signed into the system on their device

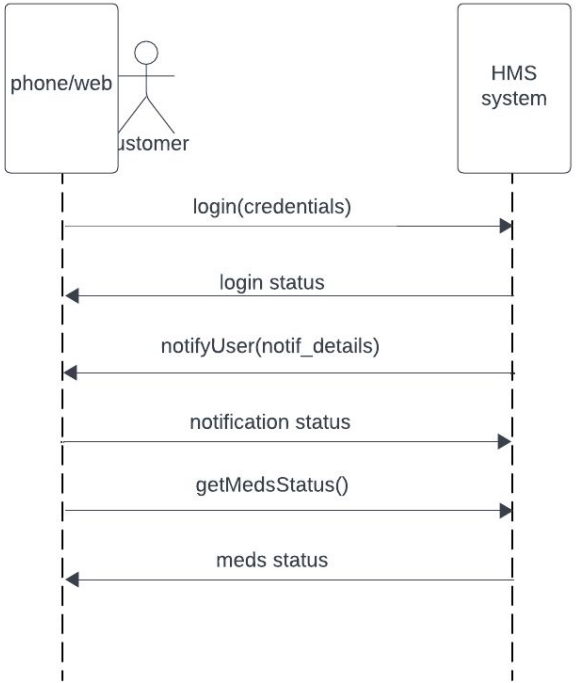
Post-conditions: System is updated with healthcare professional availability

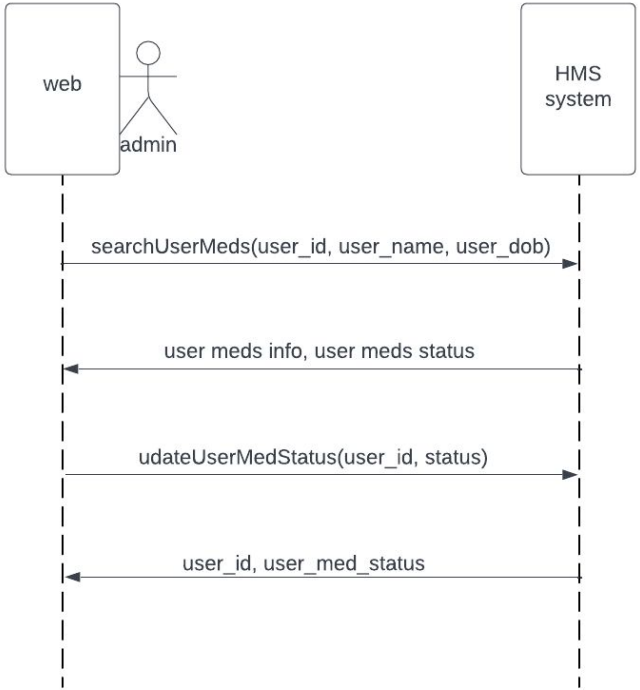
USE CASE #2 - Picking Up Medicine

Actors: patient, pharmacy admin
Description:

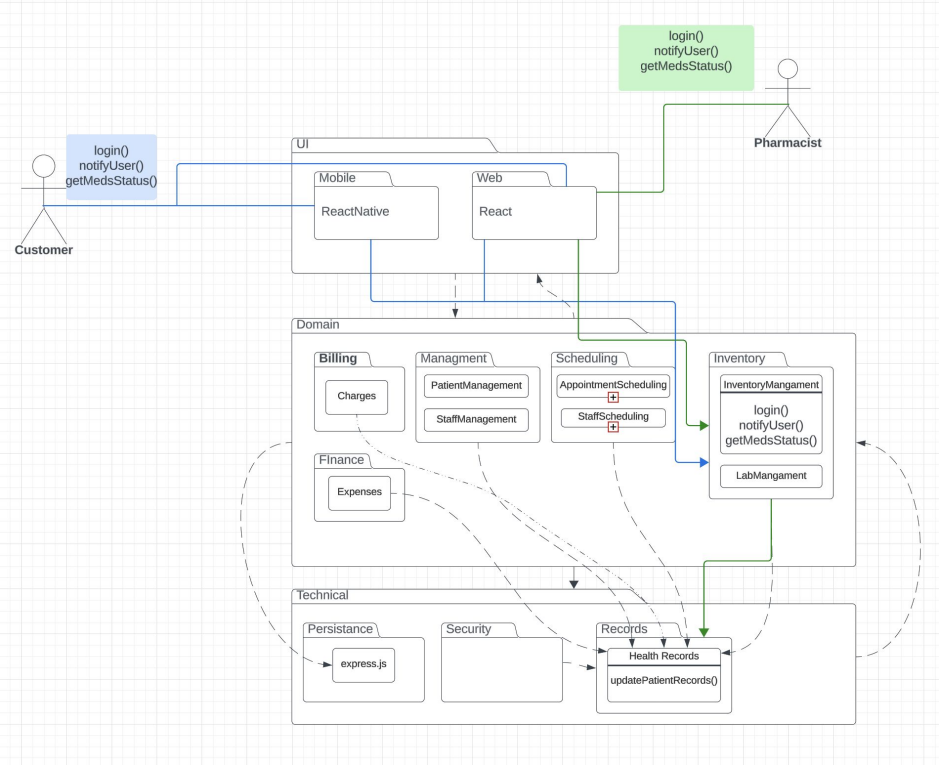
1. A **patient** receives a push-notification from the HMS on their mobile device that their prescription medication is ready for pick-up.
2. The **patient** accesses the HMS via mobile app or web portal.
3. Upon log in, the patient navigates to the “Prescription” section, which displays their approved medications and status.
4. The **patient** confirms that the medication is ready for pickup and proceeds to the hospital pharmacy.
5. At the pharmacy, the **patient** notifies the **pharmacy admin** of their arrival for medication pickup.
6. The **pharmacy admin** searches the HMS for the **patient’s** prescription using their ID, name, and DOB.
7. The HMS retrieves the prescription details and displays them to the **pharmacy admin**.
8. The **pharmacy admin** verifies that the prescriptions is approved and prepares the medication for pickup.
9. The **pharmacy admin** provides the medication to the **patient**, ensuring proper instructions are provided.
10. After dispensing the medication, the **pharmacy admin** updates the prescriptions status to “Delivered” in the HMS.
11. The HMS updates the prescriptions status in the database.

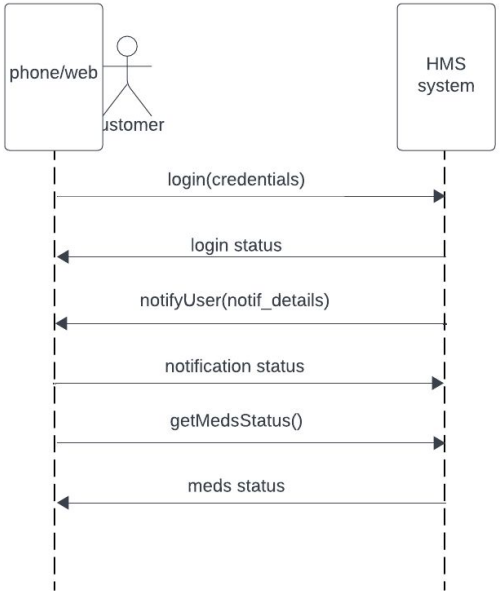
Sequence Diagram - USE CASE #2





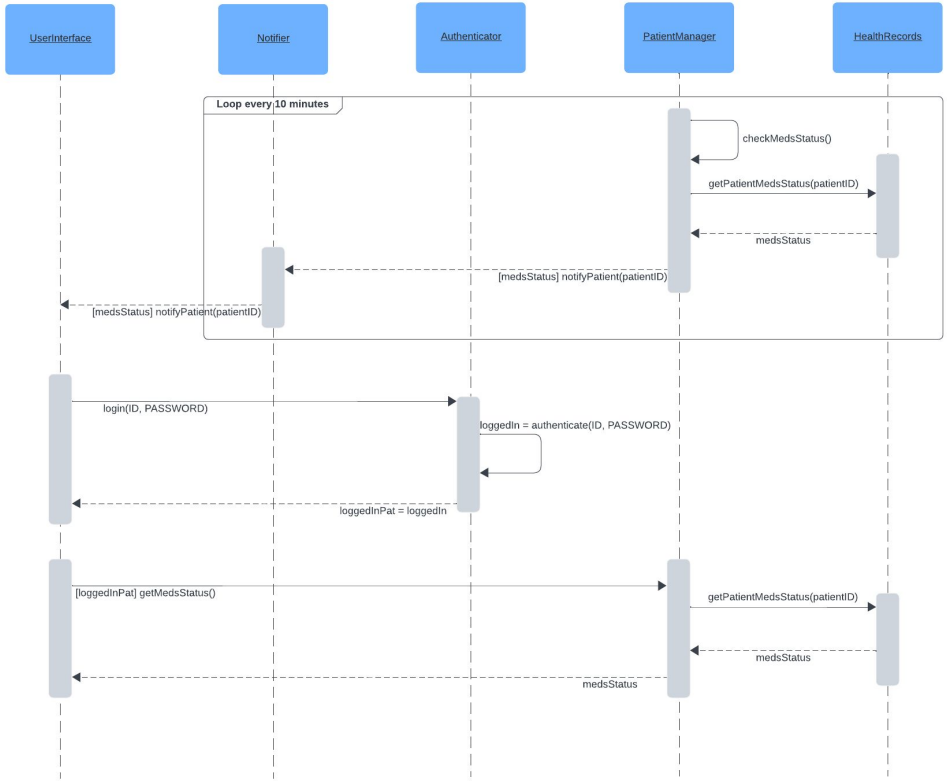
USE CASE #2 - SSD Interactions



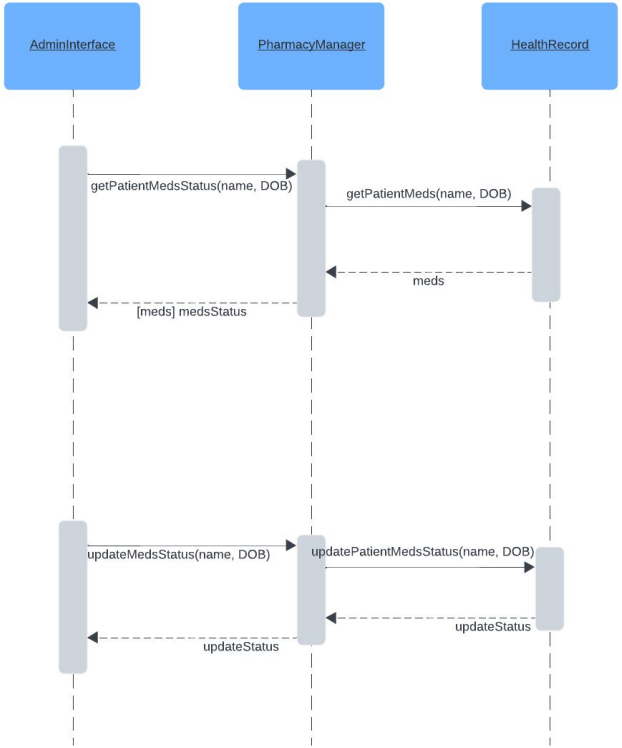


The system operations handled by the system in an SSD represent the operation calls on the Application or Domain layer from the UI layer.

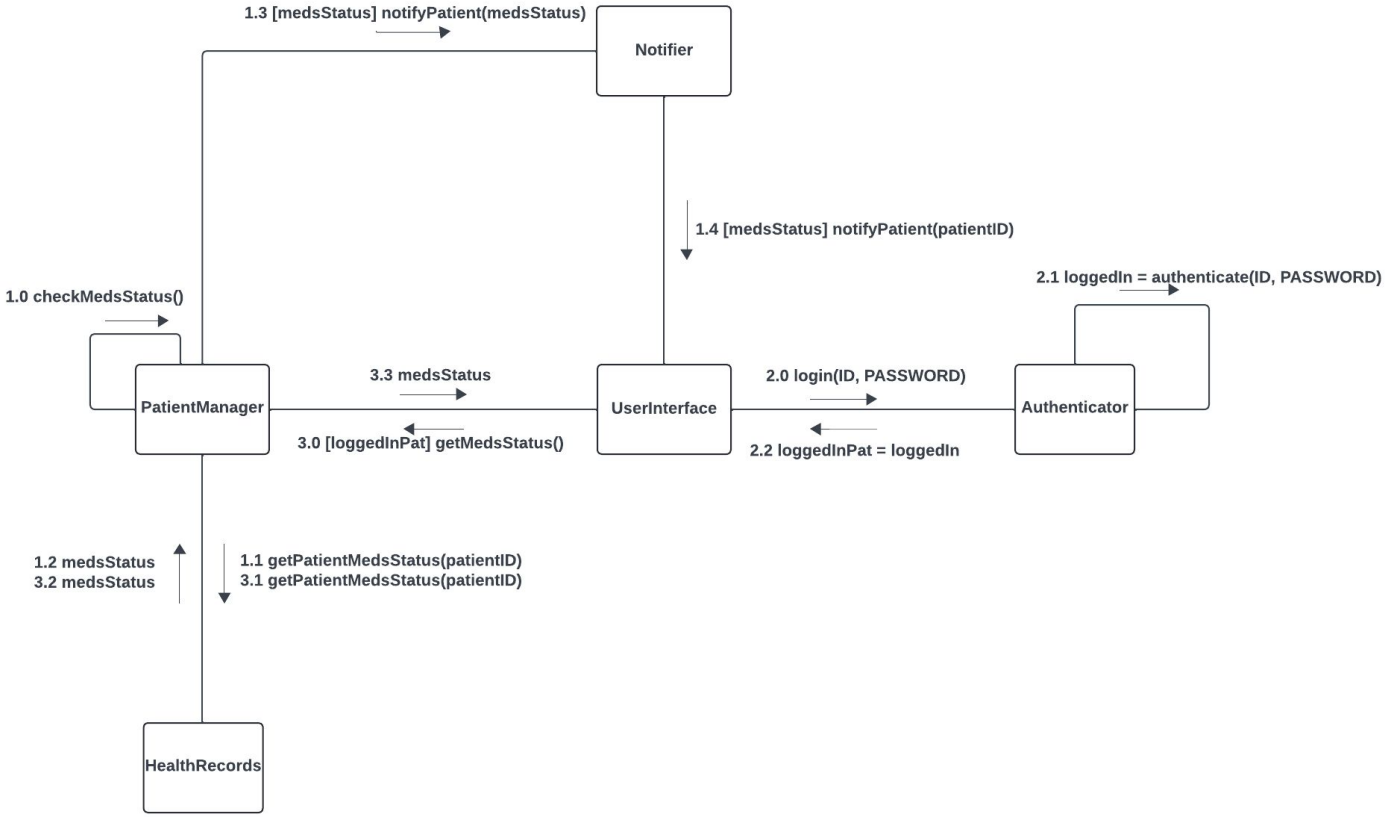
USE CASE #2 - SD


UserInterface
Notifier
Authenticator
PatientManager
HealthRecords
Loop every 10 minutes
checkMedsStatus()
getPatientMedsStatus(patientID)
medsStatus
[medsStatus] notifyPatient(patientID)
[medsStatus] notifyPatient(patientID)
login(ID, PASSWORD)
loggedIn = authenticate(ID, PASSWORD)
loggedInPat = loggedIn
[loggedInPat] getMedsStatus()
getPatientMedsStatus(patientID)
medsStatus
medsStatus

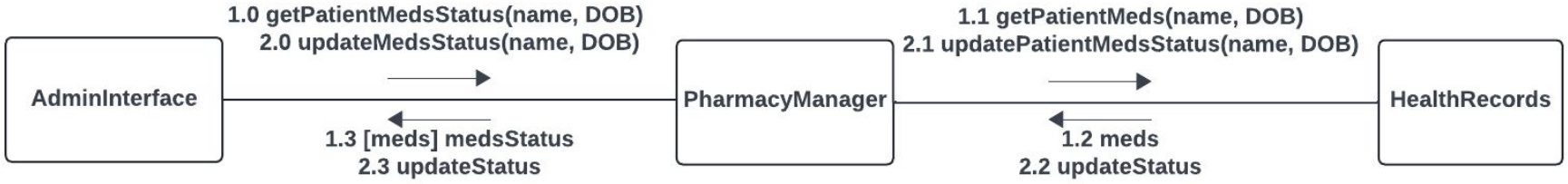
USE CASE #2 - SD


AdminInterface
PharmacyManager
HealthRecord
getPatientMedsStatus(name, DOB)
getPatientMeds(name, DOB)
meds
[meds] medsStatus
updateMedsStatus(name, DOB)
updatePatientMedsStatus(name, DOB)
updateStatus
updateStatus

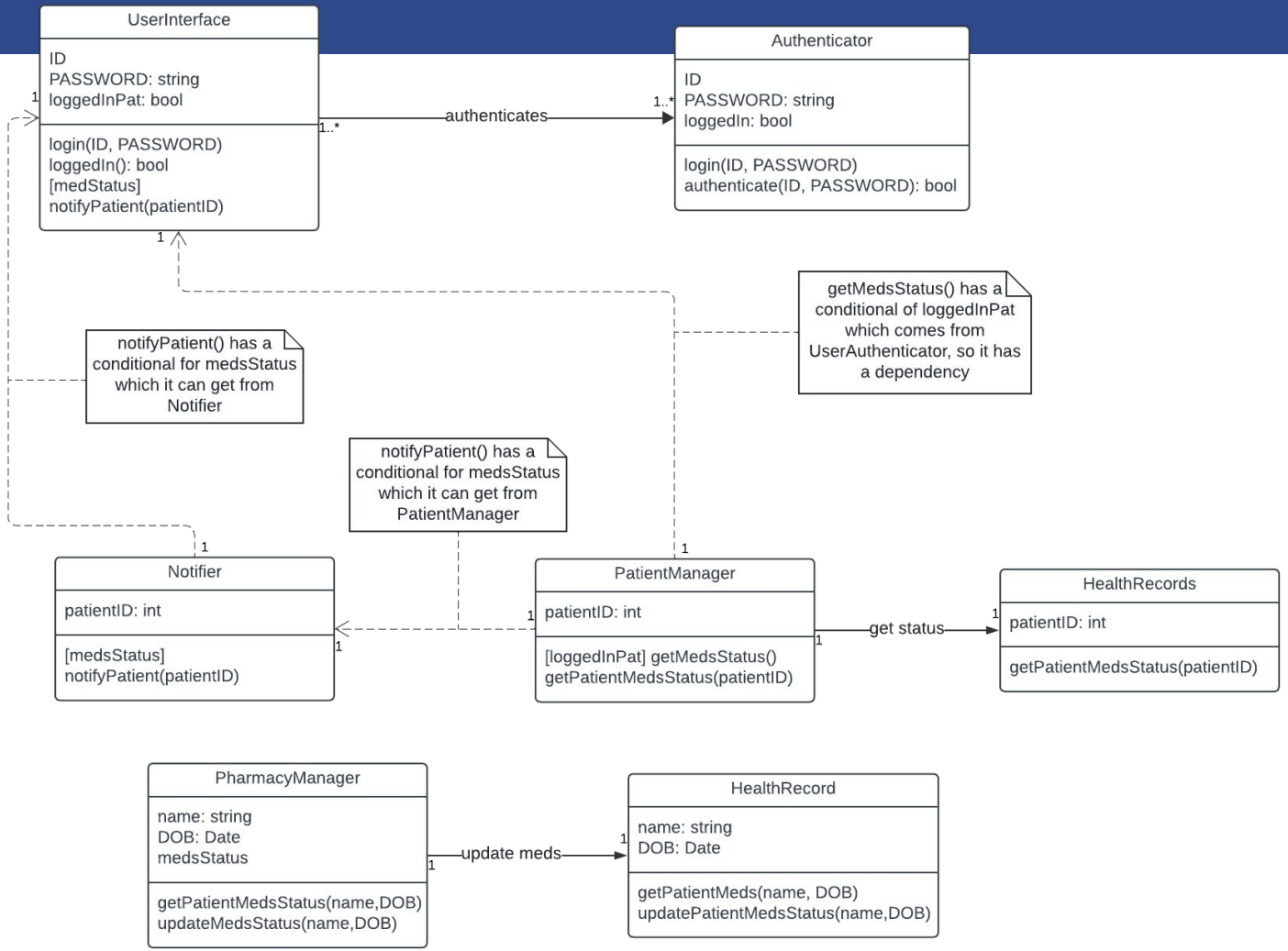
USE CASE #2 - Communication Diagrams



USE CASE #2 - Communication Diagrams



UC 2 Class Diagram



USE CASE #2 - Contract #1

Operation: updateUserMedsStatus(user_name, user_id, user_dob)

Responsibilities: The pharmacy admins updates the status of users' medications.

Cross References: Use Case #2 – Pickup Medicine

Exceptions: User not found

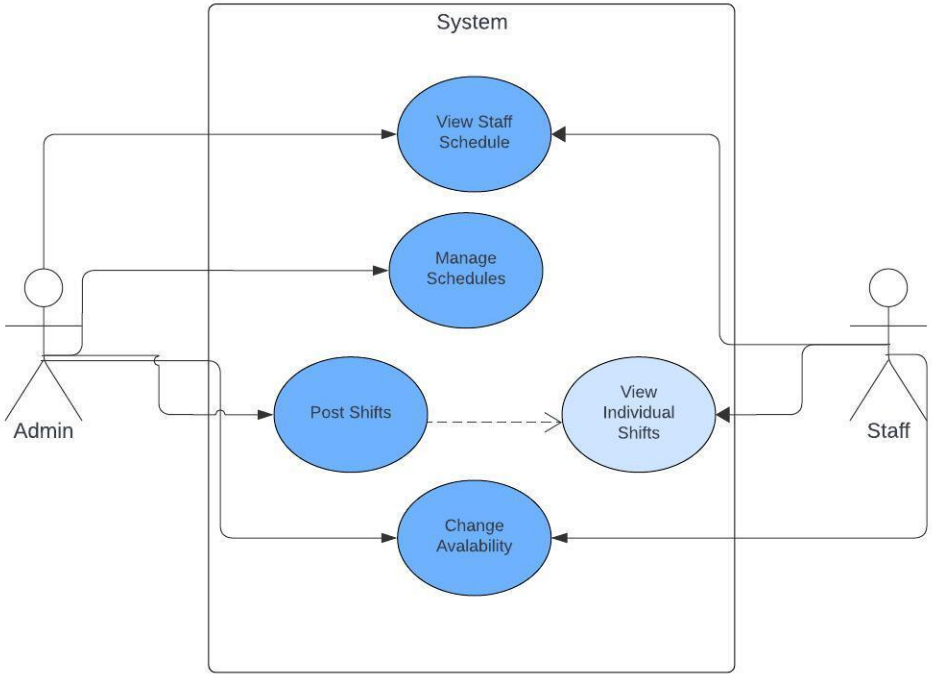
Pre-conditions: The https request has valid Authentication token and a valid body data

Post-conditions: The system updates the status of **user.medicaiton.status**

USE CASE #3-Manage Staff Schedule

Actors: Admin, Staff
Description: An **admin** (or department head) logs into the hospital management system (HMS). Upon login, they navigate to the “Staff Management” section. The system presents options to view existing **staff**, add new **staff**, update **staff** details, remove **staff**, and manage schedules. The **admin** clicks on “manage schedules”. The system displays a calendar interface where the **admin** can select the **staff** member, date and time slots for scheduling shifts, or room assignment. The **admin** fills in specific details such as shift start and end times, location/department, and any specific instruction. Upon submission, the system validates the schedule with the staff’s availabilities and updates the database with the new scheduling information. Upon validation, the system generates notifications or reminders for **staff** members regarding their scheduling information.

USE CASE #3-Manage Staff Schedule

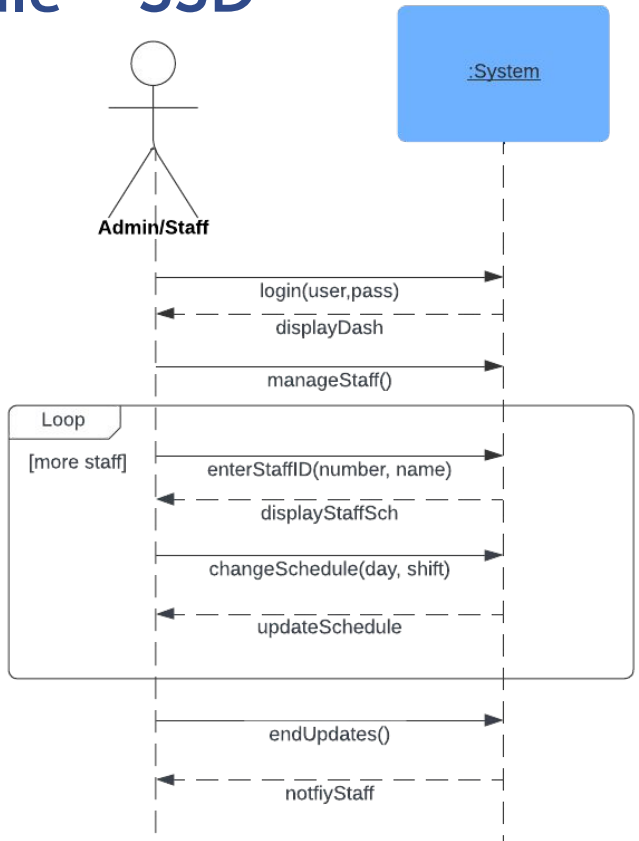


UC#3-Manage Staff Schedule - SSD

1. **Admin** logs into the hospital management system (HMS).
2. **Admin** navigate to the “Staff Management” section.
3. **Admin** enters staff name or ID number
4. **System** displays schedule,room assignment, and availabilities of that staff member
5. **Admin** updates schedule, change days working or shift start/end of that staff
6. **System** receives the updates and updates schedule, returning the changes

Repeat steps 3-6 to change different staff schedules

7. **Admin** stops updating schedule
8. **System** notifies **staff** members regarding their schedules.



USE CASE #3 - Contract #1

Operation: enterStaffID(number:int, name:string)

Responsibilities: Enter staff information to access their schedule

Cross References: Use Case #3 - Manage Staff Schedule

Exceptions: If number and name don't match or don't exist

Pre-conditions: Staff name and number must exist within the system

Post-conditions: System displays schedule of that staff

USE CASE #3 - Contract #2

Operation: changeSchedule(day, shift)

Responsibilities: The admin updates the staff's schedule

Cross References: Use Case #3 - Manage Staff Schedule

Exceptions: None

Pre-conditions: Schedule must be configured with a valid date and the shift must be between valid hours

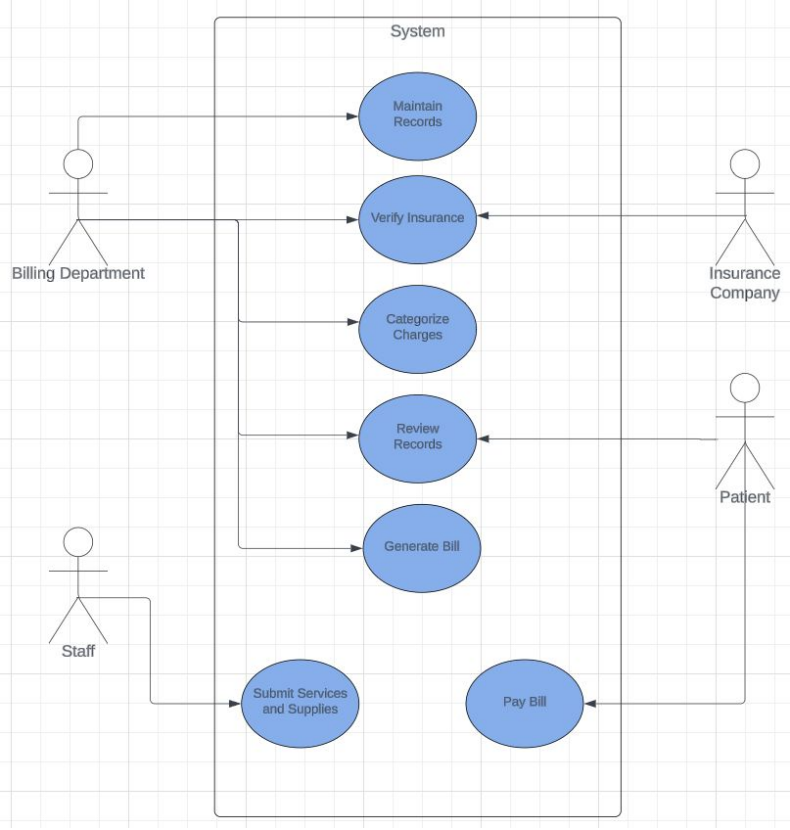
Post-conditions: System is updated with staff schedule and sends notification to affected staff

USE CASE #4 - Process Billing Patient

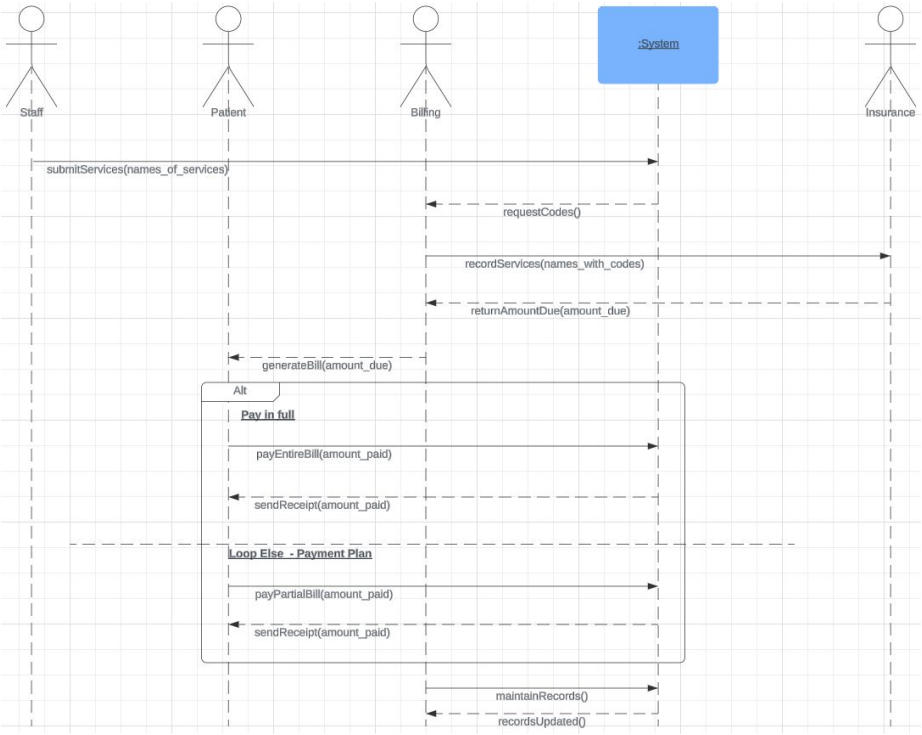
Actors: Patient, Billing Department, Insurance Company, and Staff
Description:

1. The ***Staff*** will submit all services and supplies provided such as labs, procedures, and medications.
2. The ***Billing Department*** will capture all of the charges and categorize them correctly. They will then process the claims based on the coded services entered.
3. The **Insurance Company** will send the **Billing Department** the amount not covered by insurance.
4. The ***Billing Department*** will then generate the bill for the ***patient.***
5. The ***patient*** will then review the bill and if everything is correct they will pay the bill, if necessary there can be a payment plan setup.
6. The ***Billing Department*** will then maintain records for the **patient** and report financials to the hospital.

USE CASE #4 - Process Billing Patient



UC #4 - Billing Management System - SSD



USE CASE #4 - Contract #1

Operation: submitServices(names_of_services)

Responsibilities: The staff submits the names of the services and supplies used to the system

Cross References: Use Case #4 - Process Billing Patient

Exceptions: None

Pre-conditions: Services and supplies entered must be known by system

Post-conditions: System requests codes from Billing Department

USE CASE #4 - Contract #2

Operation: payEntireBill(amount_paid)

Responsibilities: The patient pays the bill sent by the Billing Department

Cross References: Use Case #4 - Process Billing Patient

Exceptions: Payment plan could be set up instead

Pre-conditions: Billing department verifies what insurance pays for

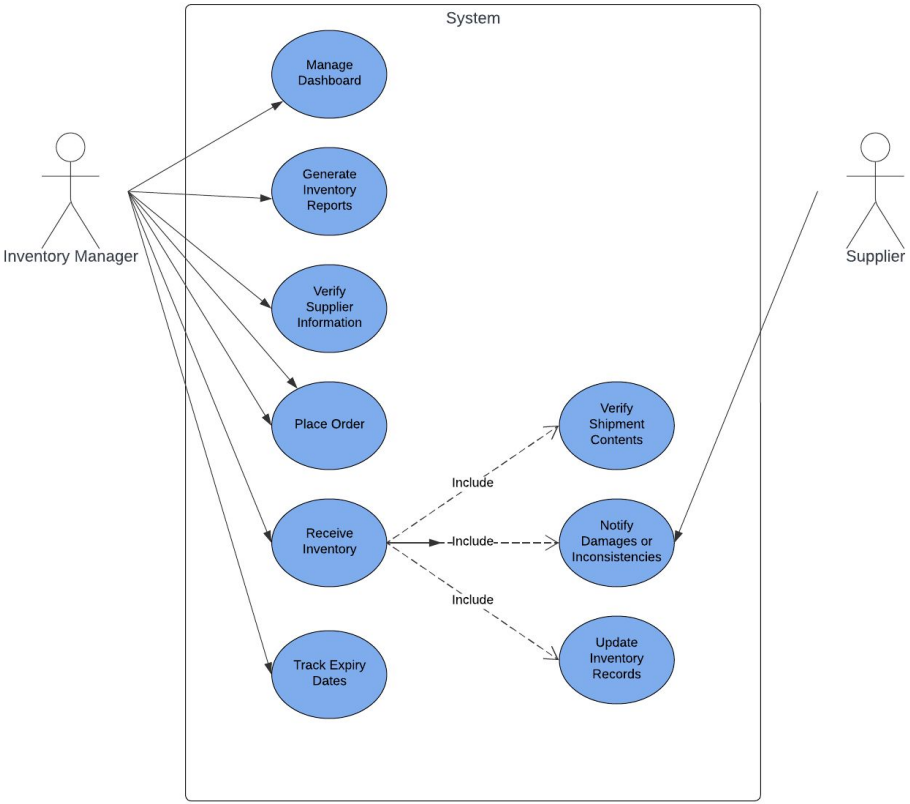
Post-conditions: Send a receipt to the patient and maintain records

USE CASE #5 - Manage Hospital Inventory

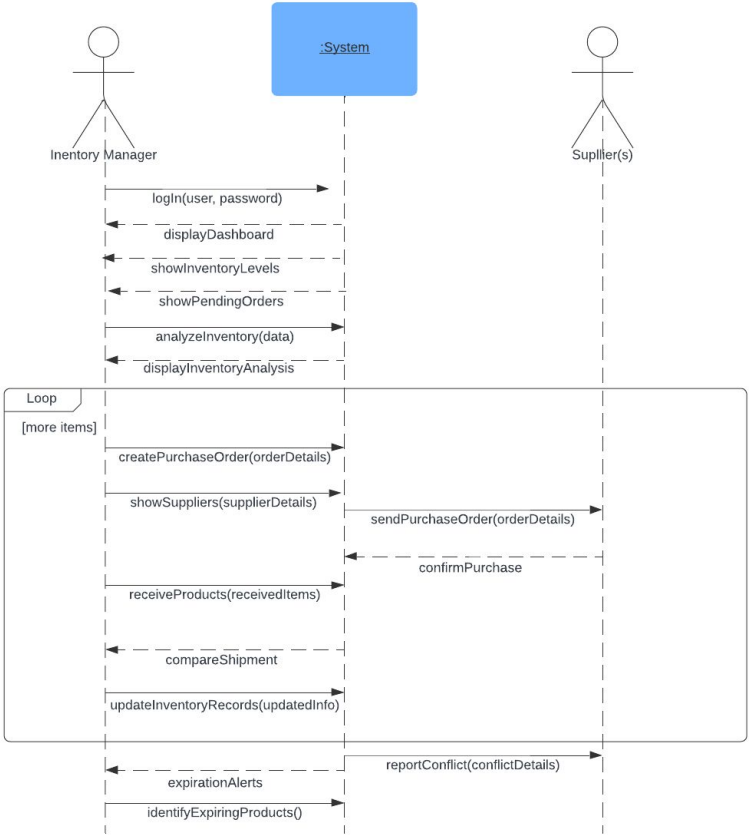
Actors: Inventory Manager, Hospital Staff, Supplier(s)

Description**:** The ***Inventory Manager*** logs into the inventory management system. The system displays a dashboard presenting current inventory levels, including quantities on hand, items on order, & items awaiting delivery. The system provides detailed reports on inventory valuation and analysis. The ***Inventory Manager*** examines inventory data, stock levels, usage trends, & reorder points[2]. The ***Inventory Manager*** generates purchase orders for supply restocking based on inventory analysis. The ***Inventory Manager*** selects ***Suppliers*** from the approved vendor list & provides details of each purchase order, including quantities, delivery dates, & any special instructions or requirements. The system generates purchase orders & sends them to the respective suppliers electronically. When shipments arrive, the ***Inventory Manager*** compares the contents to the matching purchase order on the system. The ***Supplier*** is informed of any inconsistencies or damages through the system. The ***Inventory Manager*** updates inventory records in the system to reflect the received quantities & adjustments. The system maintains stock counts, modifies inventory levels, & logs transaction information automatically. The system sends alerts for products nearing expiration, enabling timely utilization or disposal as needed. The ***Inventory Manager*** identifies products that are approaching their expiration dates using the system's expiration tracking feature.

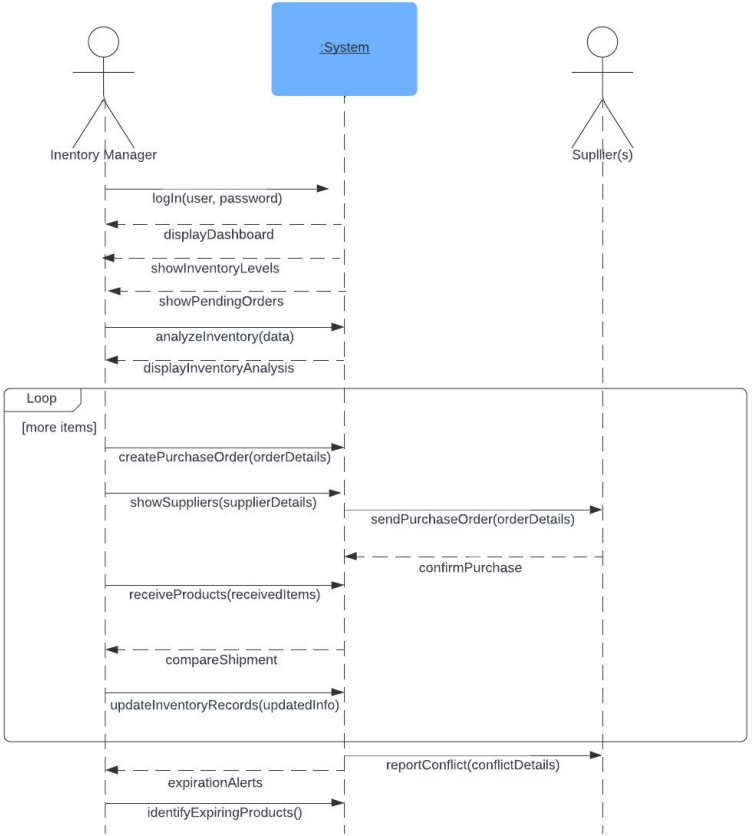
USE CASE #5 - Manage Hospital Inventory



UC#5: Manage Hospital Inventory - Sequence Diagram



UC#5: Manage Hospital Inventory - Sequence Diagram



USE CASE #5 - Contract #1

Operation: createPurchaseOrder(supplierID: integer, items: ItemList, deliveryDate: Date)

Responsibilities: To create a new purchase order in the system with the specified items, quantities, supplier, and delivery date.

Type: SW Class/Interface: Inventory System

Cross References: Use Case: Manage Hospital Inventory

Exceptions:

- ★ If supplier is not found, indicate an error.
- ★ If item is not recognized or invalid, indicate an error.
- ★ If quantity is below the minimum order quantity or exceeds the maximum order limit, indicate an error.
- ★ If the delivery date is too soon based on the supplier's capabilities, indicate an error.

Output:

- ★ A confirmation message with the purchase order ID, indicating the purchase order was successfully created.
- ★ A detailed purchase order document that can be sent to the supplier and shared with necessary departments.

Pre-conditions:

- ★ The supplierID must be an active and validated supplier in the system.
- ★ Each item in the items list must be a valid inventory item.

Post-conditions: A new purchase order is created in the system with a unique purchase order ID.

USE CASE #5 - Contract #2

Operation: identifyExpiringProducts()

Responsibilities: To identify products that are approaching their expiration dates using the system's expiration tracking feature.

Type: SW Class/Interface: Inventory System

Cross References: Use Case: Manage Hospital Inventory

Exceptions: If no products nearing expiration are found.

Output: A list of products that are approaching their expiration dates.

Pre-conditions: There are products in the inventory database that are nearing their expiration dates.

Post-conditions:

- ★ Identifying products that are approaching their expiration dates using the system's expiration tracking feature successfully.
- ★ Notifications are triggered based on the identification results, informing necessary people and departments.

GRASP Principles

1. Creator Principal:
    a. The **Authenticator** class uses a user or staff member's ID and Password to login/authenticate them into the system. It allows them to access the necessary information and resources available for them such as **StaffManagment**, **StaffScheduling**, **InventoryManagement**, **PharmacyManagement**, **Billing**, **Finance**, **LaboratoryManagement** and **PatientManagement.**
2. Controller Principal:
    a. **HealthRecords** is what every class in the system relies on. It oversees **StaffManagment**, **StaffScheduling**, **InventoryManagement**, **PharmacyManagement**, **Billing**, **Finance**, **LaboratoryManagement** and **PatientManagement.**
3. Low Coupling:
    a. The **PharmacyManagement** class operates independently without dependencies, except for its reliance on the HealthRecords class.
4. High Cohesion:
    a. **HealthRecords** handle all of the hospital records that are and can be documented.
5. Polymorphism Principle:
    a. Different types of users (Doctors, Nurses, Admins) all inherit the same class to access the same features which would be different from patients.

GRASP Principles cont.

6. Indirection Principle:

   The **AppointScheduling** class can be considered as an intermediate class as it communicates between **PatientManagement** and **StaffManagement**. Both patient and staff are able to input their availability, and an appointment scheduling class acts as an intermediate to help produce an appointment. This reduces direct dependencies between classes.

7. Pure Fabrication Principle:

   In a Health Management System, a **NotificationManager** class could be responsible for managing various notifications. This would be able to centralize all notification functions so resources in other classes can be reallocated elsewhere needed, allowing for flexibility with notifications.

8. Information Expert Principle:

   The **AppointmentScheduling** class uses the **PatientManagment** and **StaffManagment** to produce an appointment.

   Billing uses the **HealthRecords** and **Insurance** to bill patients.

SOURCES

[1] Fredricks, Asher. 53 Essential Healthcare Staff Statistics & Trends in 2024, MakeShift, 17 Jan. 2024, blog.makeshift.ca/healthcare-staff-statistics.

[2]NetSuite.com. “Reorder Point Defined: Formula & How to Use.” *Oracle NetSuite*, www.netsuite.com/portal/resource/articles/inventory-management/reorder-point-rop.shtml.

ANY
QUESTIONS?

THANK
YOU